02141 Computer Science Modelling
Context-Free Languages

CFL3: Abstract Syntax and Parser Generators

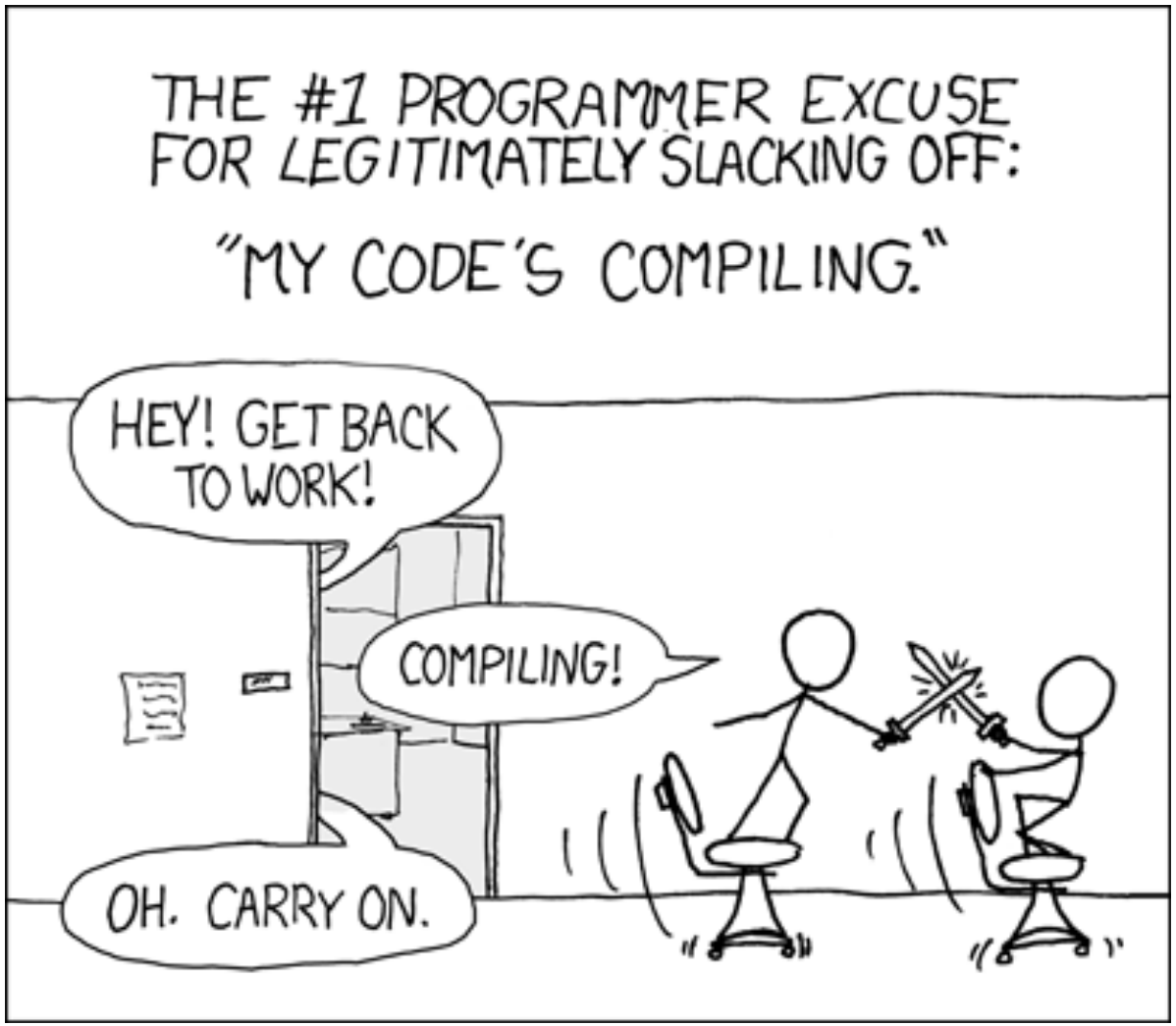
©Michael J.A. Smith, Sebastian Mödersheim

Last Time on Context-Free Languages...

- Semi-structured data formats (e.g. XML, JSON) are often context-free languages!
- A grammar called **ambiguous** if there exists more than one **parse tree** for a word of the language.
- Ambiguities lead to problems when working with the parsed string. Always try to resolve them!
- Often we can resolve ambiguities without changing the language:
  - stratification to enforce precedences
  - limiting recursion to enforce associativity
- There are some context-free languages that are **inherently ambiguous**: there is no unambiguous grammar for the language.

©Michael J.A. Smith, Sebastian Mödersheim

Parsers and Compilers

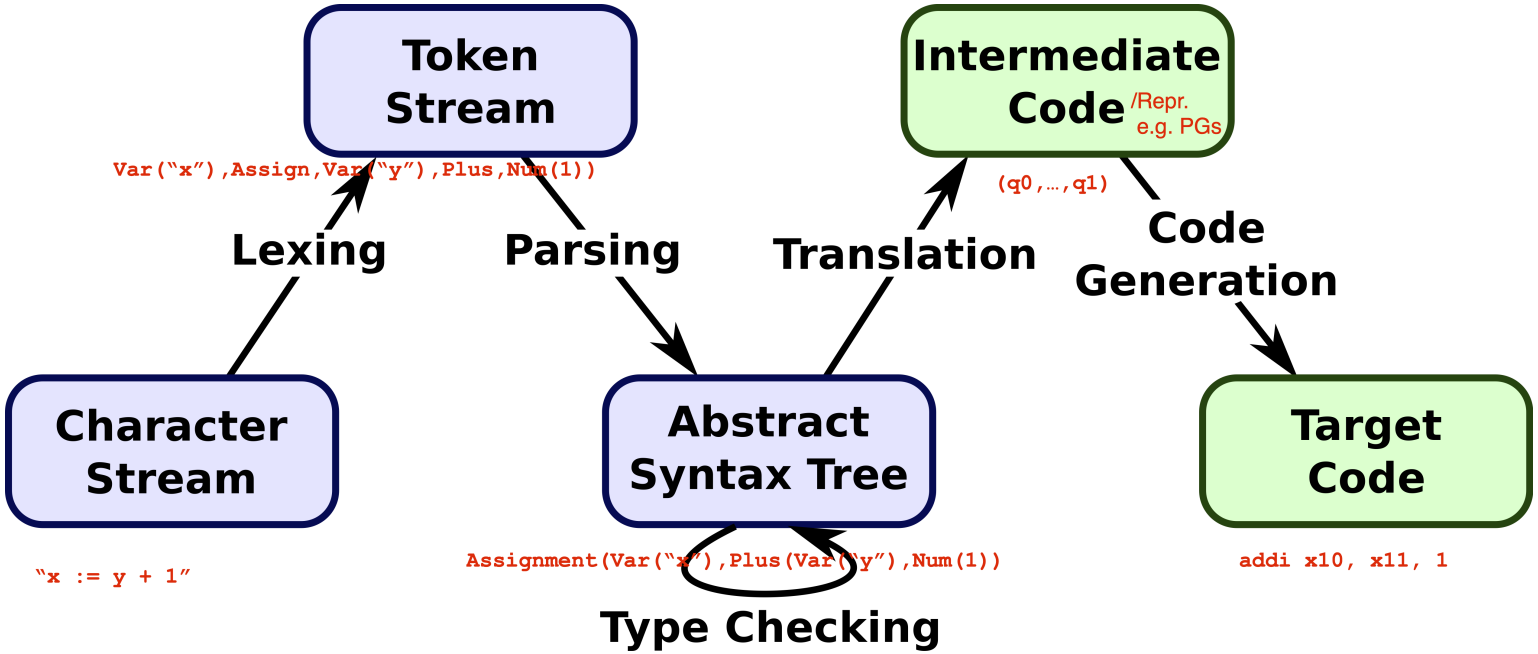


http://xkcd.com/303/

©Michael J.A. Smith, Sebastian Mödersheim

Parsers and Compilers

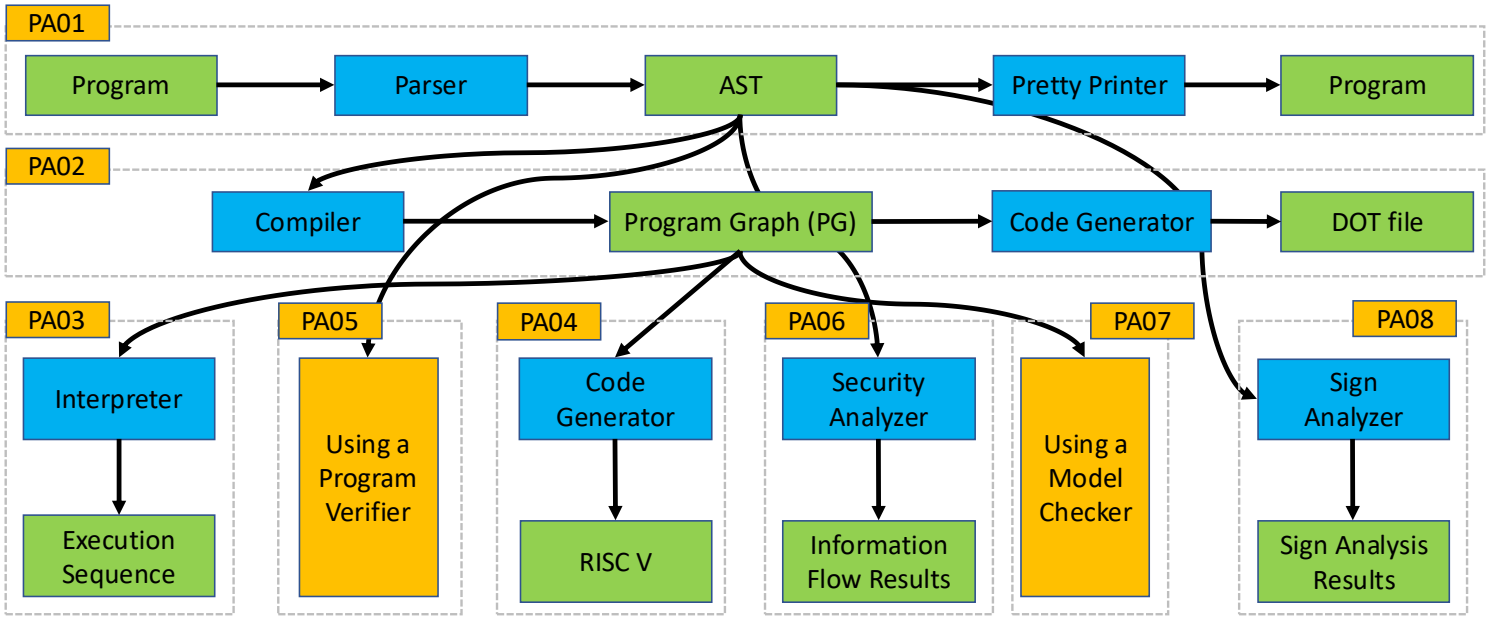
The typical workflow of a compiler looks like this



Interpreters, type checkers, program analyzers, model checkers, and all that very often have a similar workflow.

©Michael J.A. Smith, Sebastian Mödersheim

The Mandatory Assignment framework



The mandatory assignment has a similar structure.

NOTE: The `Parser` encompasses the `Lexer`. More on this later in the demo.

©Michael J.A. Smith, Sebastian Mödersheim

Why lexing (with regular expressions)?

Grammar for simple arithmetic expressions:

$$EXPRESSION \quad \rightarrow \quad EXPRESSION + EXPRESSION \quad | \quad NUM \quad | \quad (EXPRESSION)$$

We need to specify numbers (NUM)...

With a grammar:

$$\begin{array}{rcl} NUM & \rightarrow & DIGIT\ NUM \quad | \quad DIGIT \\ DIGIT & \rightarrow & 0|1|...|9 \end{array}$$

With a regular expression:

$$\begin{array}{rcl} NUM & \rightarrow & DIGIT^{+} \\ DIGIT & \rightarrow & 0|1|...|9 \end{array}$$

In practice regular expressions can be used to specify parts of the language, in particular things like numbers, identifiers, keyworks, etc.

©Michael J.A. Smith, Sebastian Mödersheim

Why lexing (with regular expressions)?

We can then keep our grammar like this:

$$
\begin{array}{lcl}
\mathit{EXPRESSION} & \rightarrow & \mathit{EXPRESSION\ PLUS\ EXPRESSION} \\
 & | & \mathit{NUM} \\
 & | & \mathit{LPAR\ EXPRESSION\ RPAR}
\end{array}
$$

where we specify separately (in a so-called lexer) the format of numbers *NUM*, summation operator *PLUS*, left parenthesis *LPAR* and right parenthesis *RPAR*.

This provides some modularity, e.g. if we want to use the same basic grammar with different formats: binary numbers instead of base 10 digits, square brackets instead of round brackets, etc.

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

Consider a fragment of a program:

```
if (x == 42) { y = "42 is the answer"; }
```

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

Consider a fragment of a program:

```
if (x == 42) { y = "42 is the answer"; }
```

To the computer, this is just a **stream of characters**:

```
i f ␣ ( x ␣ = = ␣ 4 2 ) ␣ { ␣ y ␣ = ␣ "
4 2 ␣ i s ␣ t h e ␣ a n s w e r " ; ␣ }
```

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

Consider a fragment of a program:

```
if (x == 42) { y = "42 is the answer"; }
```

To the computer, this is just a **stream of characters**:

```
i f ␣ ( x ␣ = = ␣ 4 2 ) ␣ { ␣ y ␣ = ␣ "
4 2 ␣ i s ␣ t h e ␣ a n s w e r " ; ␣ }
```

We want to convert this into a **stream of tokens**, which are more meaningful:

```
IF LPAREN ID("x") EQ NUM(42) RPAREN LBRACE ID("y") ASSIGN
      STRING("42 is the answer") SEMICOLON RBRACE
```

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

Consider a fragment of a program:

```
if (x == 42) { y = "42 is the answer"; }
```

To the computer, this is just a **stream of characters**:

```
i f ␣ ( x ␣ = = ␣ 4 2 ) ␣ { ␣ y ␣ = ␣ "
4 2 ␣ i s ␣ t h e ␣ a n s w e r " ; ␣ }
```

We want to convert this into a **stream of tokens**, which are more meaningful:

```
IF LPAREN ID("x") EQ NUM(42) RPAREN LBRACE ID("y") ASSIGN
      STRING("42 is the answer") SEMICOLON RBRACE
```

This is the job of the lexer!

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens...

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`
- Identifiers: `[A-Za-z][A-Za-z0-9]*`

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`
- Identifiers: `[A-Za-z][A-Za-z0-9]*`
- Keywords: `if`, `while`, etc.

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`
- Identifiers: `[A-Za-z][A-Za-z0-9]*`
- Keywords: `if`, `while`, etc.
- Whitespace: `[ \t\r\n]+`

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`
- Identifiers: `[A-Za-z][A-Za-z0-9]*`
- Keywords: `if`, `while`, etc.
- Whitespace: `[ \t\r\n]+`

We **match** a regular expression from the character stream, and then create a **token** from it (or discard it).

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`
- Identifiers: `[A-Za-z][A-Za-z0-9]*`
- Keywords: `if`, `while`, etc.
- Whitespace: `[ \t\r\n]+`

We **match** a regular expression from the character stream, and then create a **token** from it (or discard it). Note that `*` and `+` are interpreted **greedily**.

©Michael J.A. Smith, Sebastian Mödersheim

Lexing: From Characters to Tokens

We can identify tokens... using **regular expressions**!

Examples:

- Positive integers: `[0-9]+`
- Identifiers: `[A-Za-z][A-Za-z0-9]*`
- Keywords: `if`, `while`, etc.
- Whitespace: `[ \t\r\n]+`

We **match** a regular expression from the character stream, and then create a **token** from it (or discard it). Note that `*` and `+` are interpreted **greedily**.

The lexer provides a function `lex()` which returns the next token from the input stream. This is used by the **parser**.

©Michael J.A. Smith, Sebastian Mödersheim

Parser Generators

Writing a parser may be challenging:

- Simple parsing algorithms work only on a limited subset of grammars, and hence require you to transform your beautiful grammar into an ugly one, hence hampering understandability, extendability, etc.
- Parsing algorithms that admit (almost) any practical grammar are very sophisticated, and error-prone.

©Michael J.A. Smith, Sebastian Mödersheim

Parser Generators

Writing a parser may be challenging:

- Simple parsing algorithms work only on a limited subset of grammars, and hence require you to transform your beautiful grammar into an ugly one, hence hampering understandability, extendability, etc.
- Parsing algorithms that admit (almost) any practical grammar are very sophisticated, and error-prone.

Parser generators can save your day!

©Michael J.A. Smith, Sebastian Mödersheim

Parser Generators

Writing a parser may be challenging:

- Simple parsing algorithms work only on a limited subset of grammars, and hence require you to transform your beautiful grammar into an ugly one, hence hampering understandability, extendability, etc.
- Parsing algorithms that admit (almost) any practical grammar are very sophisticated, and error-prone.

Parser generators can save your day!

A parser generator is a tool that, given a grammar, automatically generates the (lexer and) parser for you.

There are toons of parser generators (see `https://en.wikipedia.org/wiki/Comparison_of_parser_generators`)

©Michael J.A. Smith, Sebastian Mödersheim

Two species of parser generators

The ANTLR family of generators of top-down parsers.

The yacc family of generators of bottom-up parsers: `FsLexYacc`, `bison`, etc.

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax

If we want to do something with the parsed string it is very convenient to store it in a suitable data structure, often called **abstract syntax**:

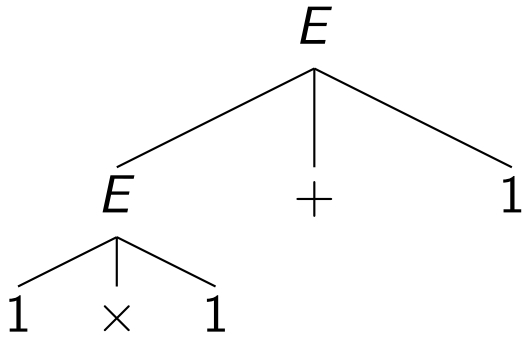
- A data structure that holds the relevant aspects of the parsed information;
- In particular, this data structure must represent the structure of the data and its contents;
- This is usually of a tree shape (hence the name AST), e.g. when parsing an arithmetic expression.
- Abstract syntax may not be a tree, e.g. when parsing a list we could just use a suitable list data structure.
- ASTs are similar to parse trees, but usually throw away irrelevant syntactic details (parenthesis, syntactic sugar, derived operators, etc.).

The original string and its parse tree(s) are often called concrete syntax.

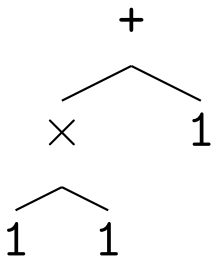
©Michael J.A. Smith, Sebastian Mödersheim

Abstract and Concrete Syntax

Example of a parse tree:



Removing irrelevant details we obtain the abstract syntax tree:



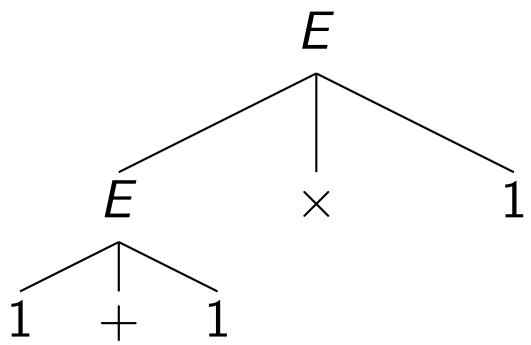
©Michael J.A. Smith, Sebastian Mödersheim

Abstract and Concrete Syntax

©Michael J.A. Smith, Sebastian Mödersheim

Abstract and Concrete Syntax

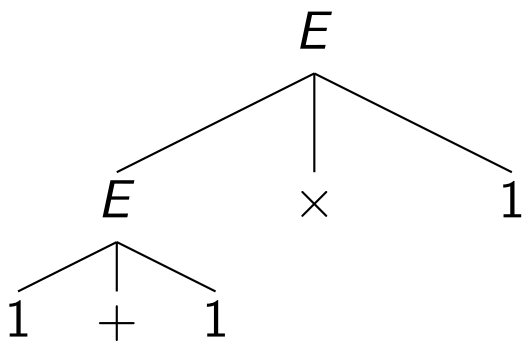
If $\times$ has higher precedence than $+$, string $1 + 1 \times 1$ should **not** be parsed as:



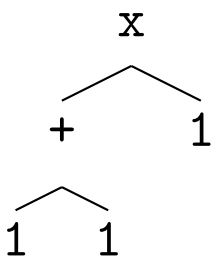
©Michael J.A. Smith, Sebastian Mödersheim

Abstract and Concrete Syntax

If $\times$ has higher precedence than $+$, string $1 + 1 \times 1$ should **not** be parsed as:



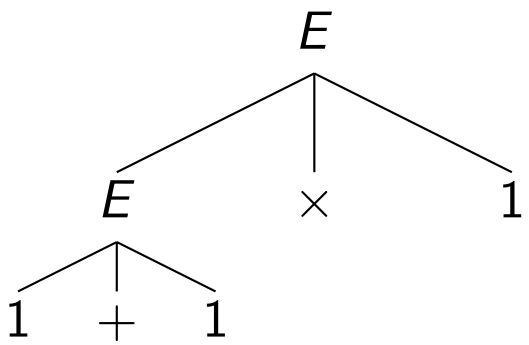
Can we ever have this abstract syntax tree?



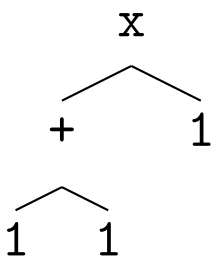
©Michael J.A. Smith, Sebastian Mödersheim

Abstract and Concrete Syntax

If $\times$ has higher precedence than $+$, string $1 + 1 \times 1$ should **not** be parsed as:



Can we ever have this abstract syntax tree?

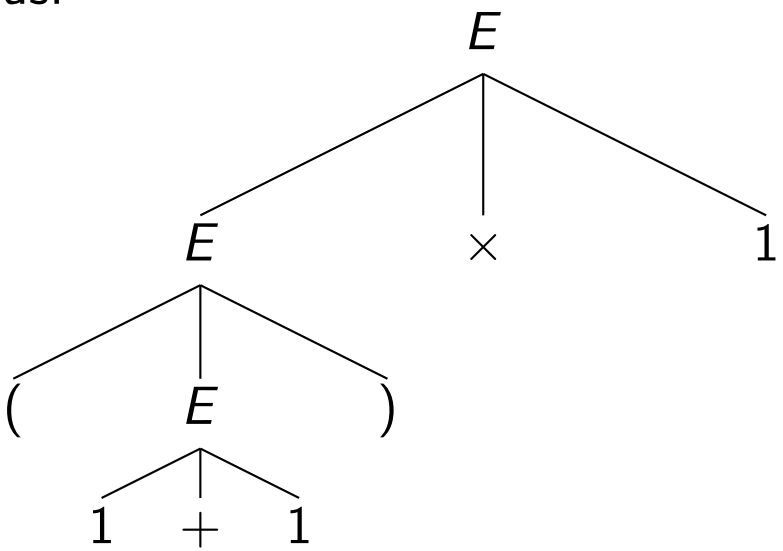


Yes we can!

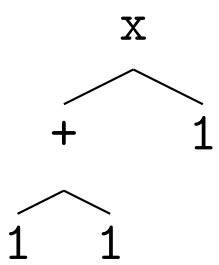
©Michael J.A. Smith, Sebastian Mödersheim

Abstract and Concrete Syntax

String $(1+1)\times 1$ should be parsed as:



This gives us the abstract syntax:

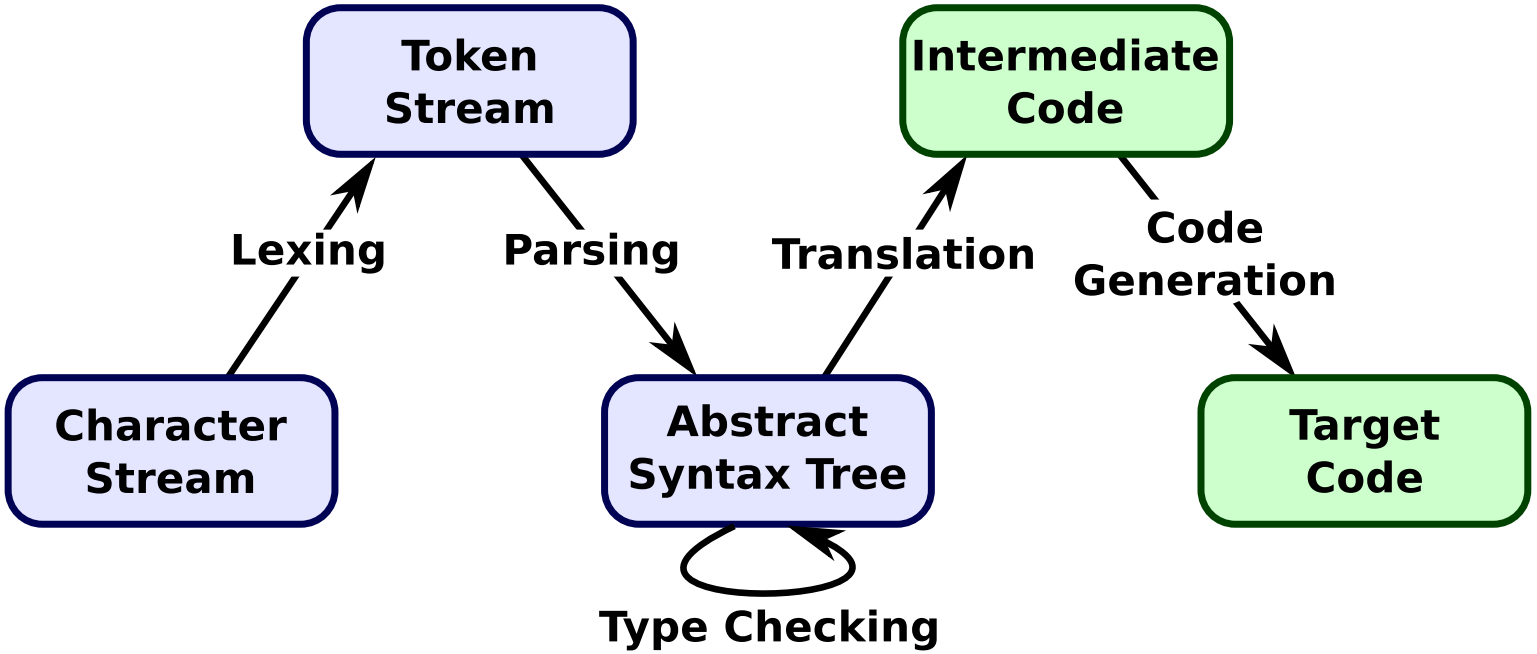


The abstract syntax is unambiguous by construction.
Ambiguity is only a problem of the concrete syntax/parsing.
Designing the abstract syntax we do not need to care about ambiguity!

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax

The typical workflow of a compiler looks like this



Interpreters, type checkers, program analyzers, model checkers, and all that very often have a similar workflow.

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for Functional Languages

In functional programming languages, abstract syntax is often based on algebraic datatypes.

Concrete Syntax

$$
\begin{array}{lll}
E \rightarrow & E + E \\
\mid & E \times E \\
\mid & N \\
\mid & (E)
\end{array}
$$

Abstract Syntax

```
type E  =  Plus of E * E
        |  Mult of E * E
        |  Num of Int
--- no data constructor for (E)
```

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Consider our grammar for arithmetic expressions

$$\begin{array}{lcl|l|l|l} E & \rightarrow & N & E + E & E \times E & (E) \\ N & \rightarrow & 0 & 1 & N0 & N1 \end{array}$$

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Consider our grammar for arithmetic expressions

$$\begin{array}{lcl|l|l|l|l} E & \rightarrow & N & E+E & E \times E & (E) \\ N & \rightarrow & 0 & 1 & N0 & N1 \end{array}$$

We implement an **abstract Java class** for expressions:

```
public abstract class ArithExpr { }
```

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Consider our grammar for arithmetic expressions

$$\begin{array}{lcl|l|l|l} E & \rightarrow & N & E+E & E \times E & (E) \\ N & \rightarrow & 0 & 1 & N0 & N1 \end{array}$$

We implement an **abstract Java class** for expressions:

```
public abstract class ArithExpr { }
```

Each production in the grammar corresponds to a **subclass** of `ArithExpr`.

```
public class PlusExpr extends ArithExpr {
    private ArithExpr expression1;
    private ArithExpr expression2;

    public PlusExpr(ArithExpr e1, ArithExpr e2) {
        expression1 = e1;
        expression2 = e2;
    }
}
```

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Consider our grammar for arithmetic expressions

$$\begin{array}{lcl|l|l|l|l} E & \rightarrow & N & E+E & E \times E & (E) \\ N & \rightarrow & 0 & 1 & N0 & N1 \end{array}$$

We implement an **abstract Java class** for expressions:

```
public abstract class ArithExpr { }
```

Each production in the grammar corresponds to a **subclass** of `ArithExpr`.

```
public class NumExpr extends ArithExpr {

    private int value;
    public NumExpr(int v) {
        value = v;
    }

}
```

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Consider our grammar for arithmetic expressions

$$\begin{array}{lcl|l|l|l} E & \rightarrow & N & E+E & E \times E & (E) \\ N & \rightarrow & 0 & 1 & N0 & N1 \end{array}$$

We implement an **abstract Java class** for expressions:

```
public abstract class ArithExpr { }
```

No new subclass for $E \rightarrow (E)$ why?

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Consider our grammar for arithmetic expressions

$$\begin{array}{lcl|l|l|l} E & \rightarrow & N & E+E & E \times E & (E) \\ N & \rightarrow & 0 & 1 & N0 & N1 \end{array}$$

We implement an **abstract Java class** for expressions:

```
public abstract class ArithExpr { }
```

No new subclass for $E \rightarrow (E)$ why?

Parentheses are part of the concrete syntax to structure terms.
In the abstract syntax, the `ArithExpr` data type is already structured!

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Example: $1 + 1 \times 1$ in the **abstract syntax**:

```
ArithExpr e = new PlusExpr(  new NumExpr(1),
                             new MultExpr(  new NumExpr(1),
                                            new NumExpr(1)));
```

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax for OO languages

Example: $1 + 1 \times 1$ in the **abstract syntax**:

```
ArithExpr e = new PlusExpr(  new NumExpr(1),
                             new MultExpr(  new NumExpr(1),
                                            new NumExpr(1)));
```

IMPORTANT!: An **abstract syntax tree** is different from a **parse tree**.

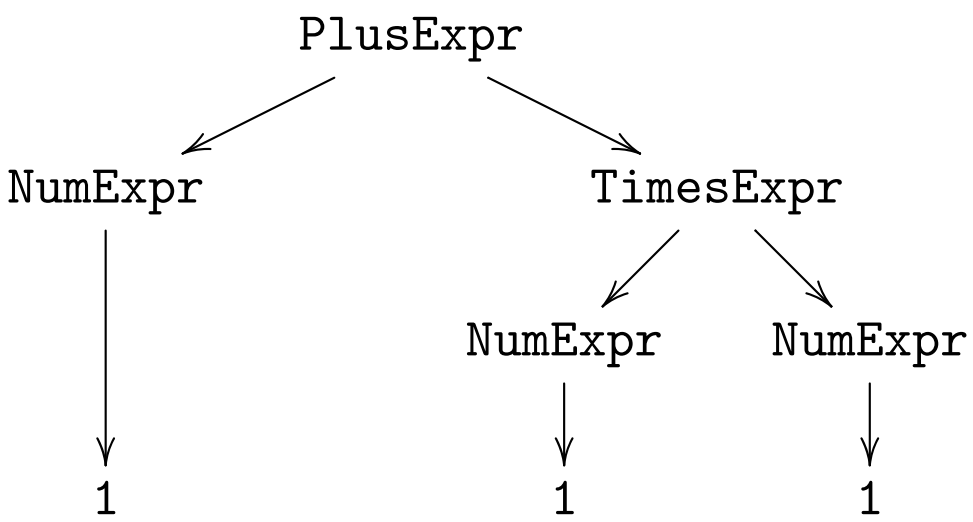
©Michael J.A. Smith, Sebastian Mödersheim

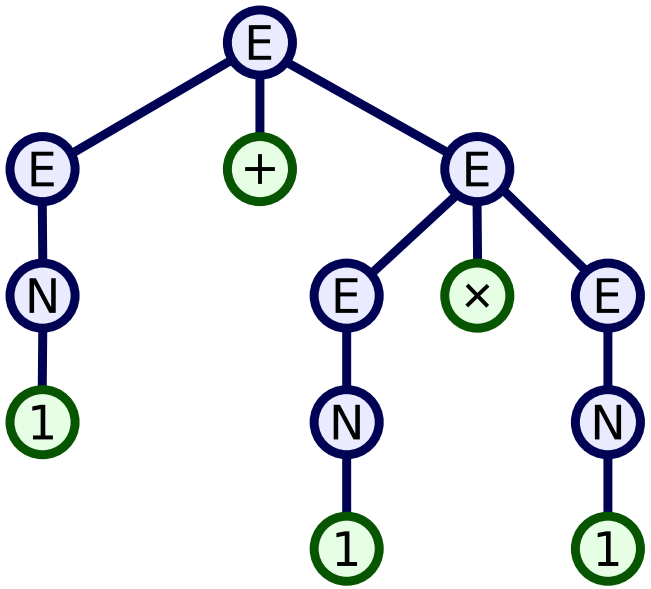
Generating Abstract Syntax for OO languages

Example: $1 + 1 \times 1$ in the **abstract syntax**:

```
ArithExpr e = new PlusExpr(  new NumExpr(1),
                             new MultExpr(  new NumExpr(1),
                                            new NumExpr(1)));
```

IMPORTANT!: An **abstract syntax tree** is different from a **parse tree**.





©Michael J.A. Smith, Sebastian Mödersheim

Exercise 3.1: abstract syntax for boolean expressions

Consider again the following grammar for boolean expressions:

$$B \rightarrow \texttt{true} \mid B \texttt{ or } B \mid \texttt{not } B \mid (B)$$

(a): Design a functional datatype that is suitable to represent abstract syntax for boolean expressions generated by the grammar, and show a statement that represents expression:

```
not ( true or true )
```

Use F# or any other functional language of your choice.

(b): Design a set of classes/structures that are suitable to represent abstract syntax for boolean expressions generated by the grammar, and show a program statement to build the abstract syntax for the above expression

Use Java or any other imperative language of your choice.

©Michael J.A. Smith, Sebastian Mödersheim

Generating Abstract Syntax

But how can we generate abstract syntax? The approach can be roughly sketched as follows:

- adopt the functional/inference/bottom-up perspective of grammars where productions $A \to \gamma(B_1, .., B_n)$ are seen as functions $\gamma : B_1 \times .. \times B_n \to A$;
- decorate the productions with code or annotations to construct the structure of type A starting from the structures obtained from B_1 .. B_n.

For example, in the grammar for arithmetic expresions the production

$$E \to E \times E$$

can be decorated (using Lex/Yacc-like notation) as

$$E \to E \ \times \ E \qquad \{\texttt{TimesExpr(\$1,\$3)}\}$$

where i indicates the i-th component of the body of the production.

©Michael J.A. Smith, Sebastian Mödersheim

What about ambiguities?

We can remove ambiguities in two ways:

1. Using fancy feature of our parser generator to indicate operator precedence and associativity. For example in Lex/Yacc:

   ```
   %left PLUS
   %left TIMES
   ```

   specifies that `TIMES` has precedence over `PLUS` and that both operators are left-associative.

2. Using the techniques seen in class (error prone, but good for exercising).

In general, it is better to rely on the features of the parser generator (which have been thoroughly tested).

©Michael J.A. Smith, Sebastian Mödersheim

Exercise 3.2

1. Go to your group's repository in gitlab.
2. Follow the instruction in the repository.
3. Complete task 0.

©Michael J.A. Smith, Sebastian Mödersheim